Englisher

Figma Design Pipeline

How the twelve Englisher screens were built into Figma from the app source — and the equivalent step-by-step workflow for doing it by hand.

Figma file	English bridge · `rKyXwnpOEf0k442mEOY8of`
Source	Duolingo-style English learning homepage (3) — 12 `.dc.html` files
Method	Figma MCP, code-to-design via Plugin API
Date	7 August 2026

PART ONE

The automated pipeline

Built programmatically through Figma's Plugin API — not captured from a rendered page. This is the record of what actually ran.

Tooling

COMPONENT	WHAT IT DID
`use_figma`	Executed JavaScript against the Plugin API inside the file. Did all the building.
`get_metadata`	Mapped existing file structure before writing anything.
`search_design_system`	Checked linked libraries for variables/styles — local APIs alone can't see remote ones.
`get_screenshot`	Visual validation after each section.
`upload_assets`	Pushed local PNG/WebP files in, returning image hashes to use as fills.
Skills	`figma-use` (API rules) + `figma-generate-design` (screen assembly).

Order of operations

This sequence *is* the method. Each step only consumes what the previous step created.

01 Discovery

`get_metadata` to map existing content, then a read-only script for local variables, components and styles, plus `search_design_system` for library assets. Result: empty design system, so one had to be built.

The same pass enumerated `listAvailableFontsAsync()` — which is how the missing Sinhala font surfaced *before* a single screen was drawn.

02 Token extraction from source

Ripgrep across the twelve `.dc.html` files for hex literals, ranked by frequency. The homepage declares the palette explicitly as named constants; the other eleven inline the same values.

03 Foundations before pixels

Variables → text styles → components. Nothing was drawn until there were tokens to bind it to. Produced 42 color tokens, 21 layout tokens, 20 text styles, 3 components.

04 Screens, section by section

One call per section. The wrapper frame is created first and appended into — never build-then-reparent, because cross-call `appendChild` fails silently and leaves orphaned frames.

05 Validate continuously

A screenshot after each section, then a final read-back audit asserting font families, empty text nodes and token-binding counts across all twelve screens.

The road not taken

`generate_figma_design` runs the opposite direction: it renders a live web app and rasterises it into Figma pixel-perfectly. Guidance treats that path as mandatory when the source contains images, since the Plugin API cannot fetch external image URLs.

A different route solved the same problem: `upload_assets` returns single-use HTTP endpoints, so the local PNGs and WebP were POSTed directly and their `imageHash` values applied as fills. Icons went in as real vectors via `createNodeFromSvg`, reusing the SVG strings already present in the source rather than redrawing them from primitives. This avoided needing a dev server for the Figma Make runtime these files depend on.

The tradeoff, stated plainly. You get instances linked to a live design system and auto-layout that reflows — but spacing is derived from reading CSS, not measured against a rendered page. The result is faithful, not pixel-verified. Serving the folder locally and running the capture alongside would close that gap.

PART TWO

The manual pipeline

The same method, rewritten for hands-on work in the Figma UI. The ordering is the part that matters — it is what stops you repainting four hundred layers later.

> **The principle:** never draw a rectangle before the token that fills it exists. Manual work drifts far more than scripted work, so the discipline has to come from ordering, not from willpower.

00 Recon

~20 min

Before drawing anything, answer three questions:

1. **What tokens does this have?** The palette is already declared as named constants in the homepage source. That is your color spec, pre-written. For everything else, search for `#` across the folder.
2. **What fonts do I have?** Type a test string in Figma *before* committing to a type ramp. This is where you would catch a missing script like Sinhala.
3. **What repeats?** Skim all twelve files and list anything appearing three or more times. That list *is* your component inventory.

01 Variables

~45 min · already done

Right sidebar → **Local variables** → new collection `Englisher/Color`. Add colors as `group/name` — the slash creates folders. Use semantic names, not literal ones: `primary/default`, never `purple-600`. Semantic names survive a rebrand.

Then a second collection `Englisher/Layout` holding `radius/*` and `space/*` as **Number** variables.

> **The step people skip.** Set **Scoping** on each variable (the gear icon beside it). Colors → Frame fill, Shape fill, Text fill, Stroke. Radii → Corner radius only. Without this, every number variable pollutes every picker and the system becomes unusable at around forty tokens.

02 Text styles

~20 min · already done

Type a line, set font, size and line-height, then right sidebar → **Style** → **+**. Name them `Display/H1`, `Body/md`.

Set line-height in **percent**, not pixels — pixel line-heights break the moment font size changes.

03 Components — start here for this file

~2 hrs

Build **one** button. Auto layout (`Shift+A`), padding, fill bound to `primary/default`, text style applied. Then:

- Duplicate → restyle as secondary → select both → **Combine as variants** (`Ctrl+Alt+K`)
- Rename the variant properties to `Variant=primary` / `Size=lg`
- Right sidebar → **+** → **Text property** → name it `Label` → click the swap icon beside the text layer's content to bind it

That final bind is what lets you retype button labels from the instance panel instead of drilling into nested layers.

Repeat for the card and the form field. **Do not start screens until this is finished** — every screen consumes these.

04 Screens

~1–1.5 hrs each

Work **outside-in**:

1. Frame at 1280 wide, fill `surface/page`, vertical auto layout, **Hug** vertical
2. Drop in empty section frames top to bottom — nav, hero, footer — each set to **Fill** horizontal
3. *Then* populate one section at a time

Inside sections, use `Shift+A` to wrap related children and set each child to **Fill** or **Hug** deliberately.

> **The most common manual mistake.** Text that should wrap needs **Fixed width + Auto height**. Leaving it on the default hug collapses or overflows the block — this was the exact bug that hit the hero headline during the automated build too.

05 Audit

~30 min

Select a whole screen frame — the right sidebar shows **Selection colors**. Any raw hex listed there is an unbound value. Click it and swap to the variable. This single panel catches nearly all token drift and is far faster than hunting layer by layer.

Then zoom to 100% and read every screen for clipped descenders and placeholder text that never got overridden.

Where manual beats scripted

Honestly: **Phase 04**. Nudging a hero until it feels right is faster by hand, and it yields judgement calls a script cannot make — optical alignment, or whether 88px of breathing room is too much.

Scripting wins on **Phases 01–02** and on any change that touches all twelve screens at once. A sensible split: generate the foundations and run bulk edits programmatically, art-direct the screens by hand.

On the time estimates. They assume the source is already decided. Add meaningful time if you are designing rather than transcribing.

APPENDIX

Reference

Extracted tokens and the one hard constraint on this project.

Core palette

Ranked by usage frequency across all twelve source files.

TOKEN	HEX	USES
`primary/default`	#6C4FF6	89
`ink/default`	#1E1B2E	30
`surface/page`	#F7F5FF	29
`surface/chip`	#ECE8FB	25
`primary/dark`	#5539E0	22
`ink/muted`	#B5AFD4	20
`ink/soft`	#6B6580	17
`accent/default`	#FFB800	12
`flame/default`	#FF6B4A	10
`success/default`	#3ECF6E	6
`teal/default`	#14D8C4	4

Typography

FAMILY	ROLE	WEIGHTS
Baloo 2	Display, headings, CTAs	Medium, SemiBold, Bold, ExtraBold
Inter	Body, labels, UI	Regular, Medium, Semi Bold, Bold
Noto Sans Sinhala	Sinhala locale	**Unavailable** — see below

Font style strings differ between the two families. Baloo 2 uses `ExtraBold` and `SemiBold` with no space; Inter uses `Extra Bold` and `Semi Bold` with a space. Guessing wrong throws at runtime.

The Sinhala constraint

The app is bilingual, but `Noto Sans Sinhala` is not present in this Figma environment — all 1,938 available families were checked, and none covers the script. Sinhala characters render as **invisible blank space**, not as tofu boxes, so the breakage is silent and easy to miss in a screenshot.

Consequently, Sinhala copy was substituted with English or romanised text, and every affected layer named `… (was Sinhala)` so the substitutions remain traceable. A Sinhala variable mode was deliberately *not* created —

adding one now would bake in broken text.

Resolving this requires uploading the font, which needs a Figma Organisation or Enterprise plan; the account used here sits on a Student-tier team, which does not support font upload.

Screens built

Homepage · Sign Up · Lesson · Exercise · Parent Dashboard · Guided Essay · Solo Essay · Formal Letter · Progress · Placement Test · Stage Lessons · Course Roadmap.

All twelve are 1280px wide, auto-layout throughout, and use only Baloo 2 and Inter. Multi-state screens were built in their default state: Placement Test as intro, Exercise as correct-answer feedback, Course Roadmap as list view.
